Code Generation

07 May, 2026

Basic blocks

A basic block is a sequence of consecutive statements in which flow of control enters at the beginning and leaves at the end without halt or possibly of branching except at the end.
Example:

$$t_1 = a * a \qquad t_2 = a * b$$
$$t_3 = 2 * t_2 \qquad t_4 = t_1 + t_3$$
$$t_5 = b * b \qquad t_6 = t_4 + t_5$$

A three-address statement $x = y(z)$ is said to *define* x and to *use* or reference y and z.
A name in a basic block is said to be live at a given point if its value is used after that point in the program, perhaps in another basic block.

Algorithm

Partitioning a sequence of three-address statements into basic blocks.

Input: A sequence of three-address statements.
Output: A list of basic blocks with each three-address statement in exactly one block.

Method:

a. Determine the set of leaders, the first statements of basic blocks. The rules we use are:
   i. The first statement is the leader.
   ii. Any statement that is the target of a conditional or unconditional goto is a leader.
   iii. Any statement that immediately follows a goto or conditional statement is a leader.
b. For each leader its basic block consists of the leader and all the statements upto but not including the next leader or the end of the program.

Example

Example 1: The following statements of source code. It computes the dot product of two vectors of length 20. A list of three-address statements performing this computation is also shown.

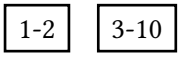
Source code:

```
begin
  prod = 0
  i = 1
  begin
    prod = prod + a * b
    i = i + 1
  do while i <= 20
end
```

Three-address code:

```
1. prod = 0 // leader by rule 1
2. i = 1
3. t_1 = a // leader by rule 2
4. t_2 = b
5. t_3 = t_1 + t_2
6. t_4 = prod + t_3
7. prod = t_4
8. t_5 = i + 1
9. i = t_5
10. if i <= 20 goto(3)
```

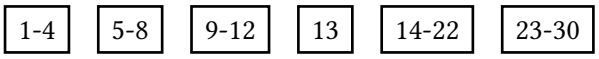
So, the basic blocks are:



Example 2:

```
 1. i = m - 1 // leader by rule 1
 2. j = n
 3. t_1 = t * n
 4. V = a[t_1]
 5. i = i + 1 // leader by rule 2
 6. t_2 = 4 * i
 7. t_3 = a[t_2]
 8. if t_3 < V goto 5
 9. j = j - 1 // leader by rule 2, 3
10. t_4 = 4 * j
11. t_5 = a[t_4]
12. if t_5 > V goto 9
13. if i >= j goto 23 // leader by rule 3
14. t6 = 4 * i // leader by rule 3
15. x = a[t6]
16. t7 = 4 * i
17. t8 = 4 * j
18. t9 = a[t8]
19. a[t7] = t9
20. t10 = 4 * j
21. a[t10] = x
22. goto 5
23. t11 = 4 * i // leader by rule 2, 3
24. x = a[t11]
25. t11 = 4 * i
26. t13 = 4 * n
27. t14 = a[t13]
28. a[t12] = t14
29. t15 = 4 * a
30. a[t_15] = x
```

So, the basic blocks are:



Transformation on basic block

A basic block constitutes a set of expressions. There are expressions whose variables are *live* on exit from the block.
The calculation of *live variables* is a matter for global flow graph. For now, we are only concerned with local transformations, and we will assume that *live variables* are known before hand.
Two basic blocks are said to be *equivalent* if they compute the same set of expressions.

Structure preserving transformations

a. **Common subexpression elimination**

Consider the basic block:

$$\begin{aligned} a &= b + c \\ b &= a - d \\ c &= b + c \\ d &= a - d \end{aligned} \quad \Rightarrow \quad \begin{aligned} a &= b + c \\ b &= b + c - d \\ c &= b + c - d + c \\ d &= b + c - d \end{aligned} \quad \Rightarrow \quad \begin{aligned} a &= b + c \\ b &= a - d \\ c &= b + c \\ d &= b \end{aligned}$$

b. **Dead code elimination**

Suppose x is dead, i.e. never subsequently used at the point where the statement $x = y$ op z appears in a basic block.

c. **Renaming temporary variables**
   # Example:

$$t = b + c$$
$$u = b + c$$

   We can replace t with u.
   # Suppose we have a statement $t = b + c$ where t is a temporary.
   # If we change this statement to $u = b + c$ where is a new temporary and change all uses of this instances of t to u. Then, the value of basic block is not changed.

d. **Interchange statements**
   # Suppose we have two adjacent statements $t_1 = b + c$ and $t_2 = x + y$.
   # We can interchange the two statements without affecting the value of the block if and only if neither x nor y is t_1 and neither b or c is t_2.

Algebraic transformation

a. Elimination of statements like $x = x + 0$ and $x = x \times 1$.
b. Replacement of statements like $x = y^2$ by $x = y * y$.

Directed Acyclic Graph (DAG)

Constructing a DAG from a three-address is a good way of determination of:
   a. common subexpression within a block.
   b. which names are used inside the block but evaluated outside the block.
   c. which statements of the block could have their computed value used outside the block.
A DAG for a basic block is a directed acyclic graph with the following labels on nodes:
   a. Leaves are labeled by unique identifiers, either variable names or constants. From the operator applied to a name, we determine whether the l-value or r-value is needed. Most leaves represent r-value. The leaves represent initial values of names and we subscript them with zero.
   b. Interior nodes are labelled by an operator symbol.
   c. Nodes are also optimally given a sequence of identifiers for labels.

Consider the following three-address code and construct the DAG.

```
 1. t1 = 4 * i
 2. t2 = a[t1]
 3. t3 = 4 * i
 4. t4 = b[t3]
 5. t5 = t2 * t4
 6. t6 = prod + t5
 7. prod = t6
 8. t7 = i + 1
 9. i = t7
10. if i <= 20 goto 1
```

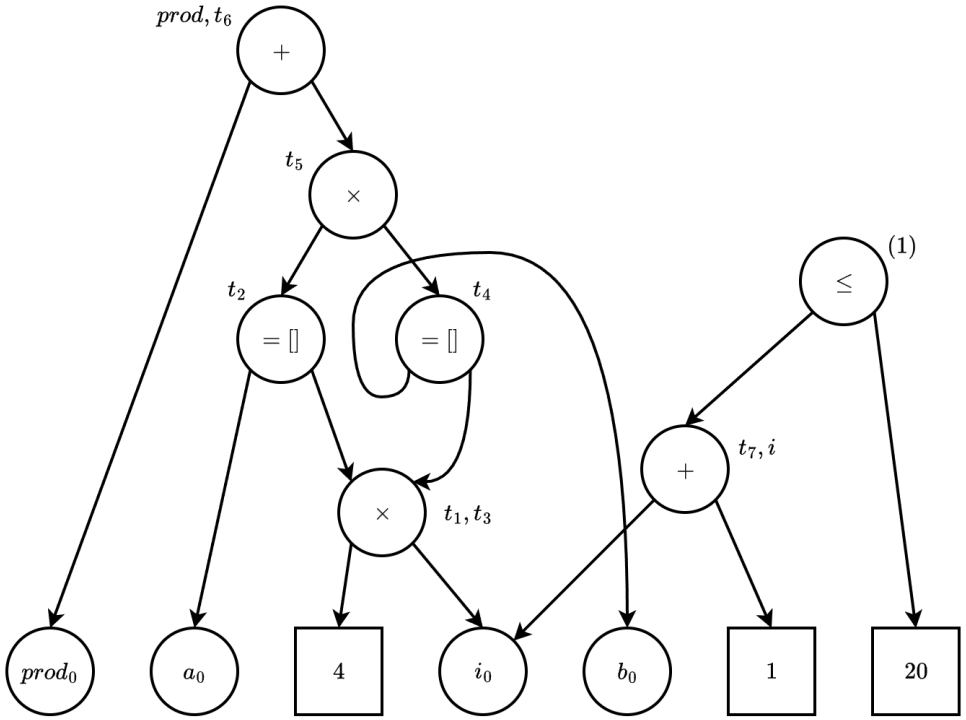


Figure 1: DAG corresponding to the three-address code above